

Assignment: Component Hunt - Instagram

1. Detailed Component Hierarchy Structure

```
<App /> (The root wrapper)
|-- <NavigationSidebar /> (Fixed left panel on desktop)
|   |-- <InstagramLogo /> (SVG Graphic)
|   |-- <NavMenu />
|       |-- <NavItem icon='home' label='Home' />
|       |-- <NavItem icon='search' label='Search' />
|       |-- <NavItem icon='compass' label='Explore' />
|       |-- <NavItem icon='film' label='Reels' />
|       |-- <NavItem icon='message' label='Messages' />
|       |-- <NavItem icon='heart' label='Notifications' />
|       |-- <NavItem icon='plus' label='Create' />
|       |-- <NavItem icon='user' label='Profile' />
|   |-- <SettingsDropdown />
|-- <MainContentLayout />
    |-- <StoriesSection /> (Horizontal scrolling container)
    |   |-- <StoryThumbnail /> (Mapped array of active stories)
    |       |-- <GradientBorder />
    |       |-- <UserAvatar />
    |       |-- <UsernameText />
    |-- <FeedContainer /> (Infinite scrolling list)
    |   |-- <PostCard /> (Individual post wrapper)
    |       |-- <PostHeader />
    |           |-- <AuthorInfo /> (Avatar + Name + Location)
    |           |-- <MoreOptionsButton /> (...)
    |       |-- <PostMediaSlider /> (Images, Videos, Carousels)
    |       |-- <PostActionsBar />
    |           |-- <LikeButton />, <CommentButton />, <ShareButton />
    |           |-- <SaveBookmarkButton /> (Aligned right)
    |       |-- <PostMetadata />
    |           |-- <LikesCount />
    |           |-- <PostCaption />
    |           |-- <CommentsPreview />
    |           |-- <Timestamp />
    |-- <RightSidePanel /> (Secondary context)
        |-- <CurrentUserSwitchProfile />
        |-- <SuggestedForYouHeader />
        |   |-- <SeeAllButton />
        |-- <SuggestionList />
            |-- <SuggestedUserCard /> (Avatar + Follow Button)
```

Assignment: Component Hunt - Instagram

`<App /> Wrapper`

`<NavigationSidebar/>`

`<Logo />`

`Home`

`Search`

`Explore`

`Reels`

`Messages`

`Notifs`

`Create`

`Profile`

`<MainContentLayout />`

`<StoriesSection />`



`User 1`



`User 2`



`User 3`



`User 4`

`<FeedContainer />`



`<AuthorInfo />`

`<PostMedia /> Image`



`Comment 1`

`Comment 2`

`Comment 3`



`<AuthorInfo />`



`Comment 1`

`Comment 2`

`<RightPanel/>`

`<SwitchProfile />`

`Suggested For You`

`<Suggested
UserCard />`

`<Suggested
UserCard />`

`<Suggested
UserCard />`

`<Suggested
UserCard />`

Component Explanations

The following pages provide a detailed paragraph explanation of every component identified in the Instagram component hierarchy. Each entry describes what the component looks like, what it does, the props or state it manages, and the design rationale behind its existence. Components are grouped by the section of the UI they belong to and classified using the Atomic Design methodology — Atoms, Molecules, Organisms, and Templates.

GROUP 1 — Navigation Sidebar

— Organism

The `NavigationSidebar` is the fixed vertical panel anchored to the left side of the screen on desktop. It contains everything a user needs for primary navigation — the brand logo, the full menu of navigation items, and a settings trigger at the bottom. On smaller screens it collapses into a bottom tab bar, demonstrating responsive component behaviour. It acts as the outermost shell that houses all nav-related child components.

Props / State: `activeRoute`, `currentUser`, `onNavItemClick`

Design Purpose: Provides a single, always-visible entry point to every major section. Encapsulating all navigation in one organism makes it easy to swap between mobile and desktop layouts without rewriting individual links.

— Atom

A pure presentational component that renders the Instagram wordmark or camera-icon SVG depending on the viewport size. On wide screens the full wordmark is shown; on narrow screens only the glyph icon appears. It is a clickable anchor that routes the user back to the home feed.

Props / State: `size`, `linkTo`, `variant` ("wordmark" | "icon")

Design Purpose: Keeping the logo as its own atom means the brand asset can be updated in one place and the change propagates everywhere it is used — not just the sidebar.

— Molecule

`NavMenu` is a vertical list that maps over a configuration array of navigation items and renders a `NavItem` for each one. It owns the logic of determining which item is currently active based on the route and passes that state down as a prop. It also handles keyboard navigation between items for accessibility.

Props / State: `items[]`, `activeRoute`, `onSelect`

Design Purpose: Extracting the list into its own molecule keeps the sidebar clean and allows the menu structure to be data-driven — adding a new section requires only a new entry in the config array, not a new JSX node.

— Atom

`NavItem` is the smallest interactive element in the sidebar. It renders an icon, a text label, and an active indicator. The icon is passed as a prop so the same atom can render Home, Search, Reels, and every other destination. When the item is active, a bold label and filled icon signal the current location. On hover, a tooltip may appear with the full label for icon-only mode.

Props / State: `icon`, `label`, `href`, `isActive`, `onClick`

Design Purpose: Using a single reusable `NavItem` atom rather than eight separate components dramatically reduces code duplication and ensures every nav destination has the same hover, focus, and active behaviour.

— Molecule

Positioned at the bottom of the sidebar, `SettingsDropdown` renders a "More" trigger button. On click it opens a popover menu with options such as Settings, Appearance, Switch Account, and Log Out. It manages its own open/close state and closes on an outside click or Escape keypress.

Props / State: `options[]`, `onSelect`, `isOpen`, `onToggle`

Design Purpose: Isolating the settings trigger into its own molecule keeps the sidebar logic clean. The dropdown pattern is reused elsewhere on Instagram (e.g., post options menu), so building it standalone enables that reuse.

GROUP 2 — Stories Section

— Organism

StoriesSection sits at the very top of the main content column and renders a horizontally scrollable row of story thumbnails. It fetches the list of accounts with active stories, maps over them to produce StoryThumbnail components, and handles the left/right scroll gesture. When a thumbnail is tapped, it triggers the full-screen story viewer overlay.

Props / State: `stories[]`, `onStoryOpen`

Design Purpose: Grouping all story-related behaviour — data fetching, scroll handling, and story launching — into one organism keeps the feed layout simple. FeedContainer never needs to know anything about how stories work.

— Molecule

Each StoryThumbnail displays a circular user avatar wrapped in a gradient ring (which turns grey once viewed) and a small username below it. It is a self-contained molecule combining three atoms: GradientBorder, UserAvatar, and UsernameText. Tapping the thumbnail calls the `onOpen` handler provided by the parent StoriesSection.

Props / State: `userId`, `avatarSrc`, `username`, `isSeen`, `onOpen`

Design Purpose: Encapsulating avatar + border + label into one molecule makes it trivial to display story indicators consistently — on the story row, in DMs, and in comment threads — all from the same component.

— Atom

A purely decorative atom that draws the iconic Instagram gradient ring (purple-to-orange) around its children. It accepts a "seen" boolean: when true it renders a flat grey border instead of the gradient, communicating that the story has already been viewed. Implemented as an SVG circle or CSS conic-gradient.

Props / State: `seen`, `size`, `children`

Design Purpose: Abstracting the gradient border into an atom means the "seen vs unseen" visual logic lives in one place and the gradient itself only needs updating once if the brand changes.

— Atom

UserAvatar renders a circular profile picture of a given size. It gracefully handles missing images by showing a default silhouette placeholder. It also supports a small "online" indicator dot for DM contexts. The image uses lazy loading so off-screen avatars do not block the page render.

Props / State: `src`, `alt`, `size`, `showOnlineIndicator`

Design Purpose: As one of the most repeated visual elements across Instagram — posts, comments, DMs, suggestions — centralising it as an atom ensures every avatar is perfectly circular, properly cropped, and accessible.

— Atom

UsernameText renders a styled, clickable username string. It navigates to the corresponding user profile on click and can optionally display a verified badge inline. Font weight and size are controlled by props so the atom works in both large headings and small caption contexts.

Props / State: `username`, `userId`, `showVerified`, `size`, `onClick`

Design Purpose: A dedicated atom prevents scattered inline link styles across the codebase and makes it easy to add global changes — e.g., a new verified badge design — in a single component.

GROUP 3 — Feed Container & Post Card

— Organism

FeedContainer is the infinite-scroll list that makes up the heart of the Instagram home screen. It requests batches of posts from the API and renders a PostCard for each one. It listens for the user scrolling near the bottom of the page to trigger the next batch fetch and shows a loading skeleton while new content arrives.

Props / State: posts[], onLoadMore, isLoading, hasMore

Design Purpose: Separating the scrolling and pagination logic from the individual post display keeps each concern isolated. PostCard never needs to know where in the list it sits, and FeedContainer never needs to know how a single post looks.

— Organism

PostCard is the main content unit of the Instagram feed. It wraps the PostHeader, PostMediaSlider, PostActionBar, and PostMetadata into a single scrollable card. It owns the like/save toggle state and orchestrates interaction between its child molecules — tapping like triggers a heart animation in the media area as well as updating the likes count in PostMetadata.

Props / State: post (id, author, media[], caption, likes, comments, timestamp), onLike, onSave, onComment, onShare

Design Purpose: Treating each post as an organism that composes smaller molecules allows Instagram to easily add new post types (Reels, product tags, collabs) by extending PostCard rather than building entirely new sections.

GROUP 4 — Post Sub-Components

— Molecule

The PostHeader sits at the top of every PostCard and combines the AuthorInfo molecule with a MoreOptionsButton on the far right. It gives the viewer immediate context about who posted and where the post originated. It does not hold local state; it simply arranges its children in a flex row with space-between alignment.

Props / State: `author (avatarSrc, username, location), onMoreOptions`

Design Purpose: Extracting the header into its own molecule means the same layout can appear on posts, Reels, and Stories headers without duplicating the avatar + username + options pattern.

— Molecule

AuthorInfo combines a UserAvatar (atom), a bold username link, and an optional location tag into a compact horizontal unit. Clicking the avatar or username navigates to the author's profile. The location text, when present, is a secondary tappable element that opens a location-tagged feed.

Props / State: `avatarSrc, username, userId, location, avatarSize`

Design Purpose: Reusing AuthorInfo across posts, comments, DMs, and suggestions ensures the user's identity is presented consistently everywhere in the app, reducing cognitive load.

— Atom

A simple "... " icon button fixed to the top-right corner of the PostHeader. On click it opens a bottom sheet or popover with context-sensitive actions: Report, Unfollow, Go to Post, Share, Embed, and Copy Link. For the post author, options change to Edit and Delete.

Props / State: `isOwner, postId, onAction`

Design Purpose: Keeping this as an isolated atom allows the options menu to be reused across posts, comments, and stories without coupling it to any specific parent component.

— Molecule

PostMediaSlider handles displaying one or more media items — photos, videos, or a mix — within a post. When multiple items exist, horizontal swipe gestures or dot indicators let the user navigate between them. Videos autoplay muted on scroll into view and unmute on tap. A carousel index badge (e.g., "2 / 5") overlays the top-right corner for multi-image posts.

Props / State: `mediaItems[] (src, type, altText), onDoubleTap`

Design Purpose: Isolating media rendering from the action bar and metadata means the same slider molecule can be dropped into Explore grids, Reels, and Profile grids with different size constraints.

— Molecule

PostActionsBar is the row of interactive icons directly below the media. On the left sit LikeButton, CommentButton, and ShareButton. On the far right, pushed by flex, is SaveBookmarkButton. Each button is an atom that emits an event upward; PostActionsBar simply lays them out and forwards the events to PostCard, which owns the actual state.

Props / State: `isLiked, isSaved, onLike, onComment, onShare, onSave`

Design Purpose: Grouping all action icons into one molecule keeps the layout contract explicit. PostCard deals with one actions-bar component, not four loose buttons, making it easy to rearrange or add actions later.

/// — Atoms

These four atoms are the individual icon-buttons in the action bar. LikeButton renders an outlined heart that fills with red and plays a bounce animation when toggled. CommentButton opens the comments sheet. ShareButton opens a share-to dialog. SaveBookmarkButton toggles between outlined and filled states. Each atom is completely self-contained visually; state is lifted to PostCard.

Props / State: isActive, onClick, size, ariaLabel

Design Purpose: Building each action as a separate atom allows Instagram to independently A/B test, reorder, or swap icons without touching any other part of the post layout.

— Molecule

PostMetadata renders everything below the action bar: the likes count, the caption (with expandable "more" truncation), a preview of the most recent one or two comments, and the relative timestamp. It is read-only and presentational. Username mentions and hashtags inside the caption are rendered as interactive inline links navigating to the relevant profile or tag feed.

Props / State: likesCount, caption, comments[] (preview), timestamp, authorUsername

Design Purpose: Grouping textual post data into one molecule ensures consistent typography treatment across all post types and makes it straightforward to add new metadata fields without restructuring PostCard.

GROUP 5 — Right Side Panel

— Organism

The `RightSidePanel` is a fixed secondary column that appears on wide desktop viewports (typically 1280 px and above). It provides contextual content — your own profile switcher at the top and a list of suggested accounts below. It is hidden on tablet and mobile. The panel does not scroll independently; it stays in place as the feed scrolls past it.

Props / State: `currentUser`, `suggestions[]`, `onFollowUser`

Design Purpose: Putting all secondary contextual content into one panel organism respects the primary feed's visual weight while still offering discovery and account management without a page navigation.

— Molecule

This molecule displays the logged-in user's avatar and username at the top of the right panel, with a "Switch" link that opens the account-switcher flow. It communicates to the user which account is currently active — critical for users who manage multiple Instagram accounts.

Props / State: `avatarSrc`, `username`, `userId`, `onSwitch`

Design Purpose: Surfacing the current account identity in the panel prevents confusion for multi-account users. Encapsulating it as a molecule makes it reusable in the mobile account-switcher sheet too.

— Molecule

A simple heading molecule that displays the text "Suggested for you" in bold with a "See all" link aligned to the right. The `SeeAllButton` atom-child navigates to a dedicated suggestions page. Despite its simplicity, keeping this as a molecule separates the header display from the list below.

Props / State: `title`, `onSeeAll`

Design Purpose: Treating headings as molecules rather than raw text nodes allows the "See all" interaction to be consistently styled and tracked for analytics wherever the pattern appears in the app.

— Organism

`SuggestionList` maps over an array of suggested accounts and renders a `SuggestedUserCard` for each. It handles the follow action, updating the card state when the user follows or dismisses a suggestion, and can request a fresh batch of suggestions when the list runs low.

Props / State: `suggestions[]`, `onFollow`, `onDismiss`

Design Purpose: Containing the list logic in its own organism keeps `RightSidePanel` clean and lets the same suggestion list appear in other contexts — Explore, post-follow confirmations, and onboarding.

— Molecule

Each `SuggestedUserCard` displays a circular user avatar, the account username, a secondary line describing why they were suggested (e.g., "Followed by username"), and a "Follow" button. Once pressed, it toggles to "Following" state. A dismiss "x" button lets the user remove the suggestion.

Props / State: `userId`, `avatarSrc`, `username`, `reason`, `isFollowing`, `onFollow`, `onDismiss`

Design Purpose: The card pattern — avatar + label + action — is one of the most repeated patterns in social apps. Keeping it standalone means it works identically in the right panel, Explore, notifications, and search.